

THS-II · EMS · Reinforcement Learning Pipeline v3.1 — TomTom Real-Route Only

Toyota Prius Gen 3 (ZVW30) · THS-II Power-Split Hybrid
PPO Agent (Stable-Baselines3) · Custom Gymnasium · ONNX / Raspberry Pi HIL
Training & Evaluation: TomTom Real-World Routes ONLY

CONFIDENTIAL

v3.1

INSAT 2025/2026

Oussama Chouaibi · EABA | Znazen Khalil · Karoui Wassim · Chouaibi Mohamed Aziz

v3.1 Key Changes — TomTom-Only Architecture

NEW VERSION

Change	v3.0 (Previous)	v3.1 (This Document)
Drive cycle sources	WLTC + FTP-75 + US06 + TomTom	TomTom Real-Routes ONLY
RL training data	Standard CSV cycles as fallback	100% TomTom route episodes
Gymnasium fallback	CSV loader as default	Removed — TomTom is mandatory
Day ordering	Phase 0 → Days 1–6 (mixed)	Fully logical sequential order
Baseline comparator	Rule-based on CSV cycles	Rule-based on TomTom routes
ONNX validation	Mixed cycle validation	TomTom route validation only
env/drive_cycles/	WLTC.csv, FTP75.csv, US06.csv	REMOVED from mandatory deliverables

BREAKING CHANGE: FTP-75 and US06 CSV drive cycles are entirely removed from v3.1. The RL agent is trained, validated, and exported exclusively on TomTom real-world routes. The Gymnasium environment no longer accepts a CSV fallback — route_cache is mandatory.

System Parameters

Parameter	Value
Vehicle	Toyota Prius Gen 3 (ZVW30) — THS-II Power-Split Hybrid
RL Algorithm	PPO — Proximal Policy Optimisation (Stable-Baselines3)
Physics Engine	modeling1.py v4 — 2ZR-FXE · PSD · MG1/MG2 · NiMH
Gymnasium	Custom THSEnv — TomTom route_cache MANDATORY (no CSV fallback)
Drive Cycles	TomTom Real Routes ONLY — 5 representative route types
Route / GPS	TomTom Traffic API — Real-time routes, segments, traffic, elevation
Drive Modes	EV · ECO · NORMAL · PWR (Prius Gen 3 factory — SPORT absent)
RL Objectives	Minimise CO2 · Fuel · Energy · Maximise Battery Lifetime
HIL Platform	Raspberry Pi 4 — 4 GB RAM [Optional Day 6]
Dashboard	Streamlit — live route map, SOC, fuel, CO2 telemetry
Day Order	Phase 0 → Day 1 Setup → Day 2 TomTom Env → Day 3 PPO Training → Day 4 Eval+ONNX → D5 CARLA → D6 CAN

Table of Contents

Phase 0	TomTom Real-Route Pipeline [MANDATORY — First Phase]
Chapter 1	Introduction & Problem Statement
Chapter 2	State of the Art
Chapter 3	System Architecture & Solution
Chapter 4	System Components & Technologies
Chapter 5	Reinforcement Learning Design
Day 1	Environment Setup & Physics Verification
Day 2	TomTom Gymnasium Environment [TomTom routes replace CSV]
Day 3	PPO Training on TomTom Routes [TomTom-only training]
Day 4	Evaluation, ONNX Export & Dashboard [TomTom validation]
Day 5	GPS Route Fine-Tuning & Predictive EMS [GPS-enriched]
Day 6	CARLA Live Co-Simulation [Optional]
Day 7	CAN Bus Hardware-in-the-Loop [Optional]
§ 13	Project Deliverables
§ 14	Conclusion & Future Work

PHASE 0

0

TomTom Real-Route Pipeline

MANDATORY FIRST PHASE — Prerequisite for all days

Phase 0 is the mandatory entry point of the entire project. Before any physics simulation or RL training, a real driving route is generated from the user's chosen Origin and Destination using the TomTom Traffic API. In v3.1, TomTom routes are the **sole** source of driving cycles — there is no CSV fallback. The route is divided into typed segments, each enriched with live traffic, speed limits, elevation, and road class. These segments become the driving cycle fed to both the physics engine and the RL agent.

P0.1 · Why TomTom — Sole Data Source in v3.1

Feature	TomTom API (v3.1 — Only Source)	ORS / OSRM (Removed)
Real-time traffic	✓ Live jam factor per segment	Limited / static
Speed profiles	✓ Historical + live speed	Speed limit only
Road class tags	✓ FRC 0-7 (motorway → path)	OSM class
ETA / travel time	✓ Per-segment ETA	✗
Elevation	Via Waypoint Snap API	Open-Elevation SRTM
RL Training use	✓ MANDATORY — sole cycle source	✗ Not used in v3.1

P0.2 · TomTom API Registration & Key Setup

- Register at **developer.tomtom.com** — free tier: 2,500 req/day.
- Enable: Routing API, Traffic Flow API, Map Display API, Waypoint Snap API.
- Store key as environment variable: `export TOMTOM_API_KEY=your_key_here`
- Never commit the key — add `.env` to `.gitignore`.

```
pip install requests geopy scipy shapely python-dotenv
```

P0.3 · Dashboard Route Input (Streamlit)

The Streamlit dashboard provides two input fields: Origin and Destination. On clicking Generate Route, the backend calls TomTom Routing API and returns a GeoJSON polyline. The route is split into segments and cached locally as the RL training cycle.

```
# app/streamlit_dashboard.py - route input widget
origin = st.text_input('Origin', 'Place de la Bastille, Paris')
dest = st.text_input('Destination', 'Aéroport CDG, Paris')
if st.button('Generate Route'):
    route = fetch_tomtom_route(origin, dest, api_key=TOMTOM_KEY)
    segments = segment_route(route) # returns List[RouteSegment]
    st.session_state['segments'] = segments
    render_folium_map(segments) # colour-coded by recommended mode
```

P0.4 · TomTom Route Fetcher (gps/route_fetcher_tomtom.py)

```
BASE = 'https://api.tomtom.com/routing/1/calculateRoute'
def fetch_tomtom_route(origin_ll, dest_ll, api_key):
    url = f'{BASE}/{origin_ll[0]},{origin_ll[1]}:{dest_ll[0]},{dest_ll[1]}/json'
    params = dict(key=api_key, traffic='true', travelMode='car',
    routeType='eco', computeTravelTimeFor='all',
    sectionType='traffic', instructionsType='coded')
    r = requests.get(url, params=params, timeout=10)
    r.raise_for_status()
    return r.json()['routes'][0] # full route object
```

P0.5 · Segment Extraction & Enrichment

Field	TomTom Source	RL Use
start / end coords	legs[].points[]	Folium map polyline

Field	TomTom Source	RL Use
length_m	summary.lengthInMeters	gps_dist_to_next_seg
travel_time_s	summary.travelTimeInSeconds	ETA display
road_class (FRC)	sections[].sectionType + FRC tag	segment_type (0/1/2)
speed_limit_kmh	guidance.instructions[].speedLimit	velocity normalisation
jam_factor [0–10]	Traffic Flow API /flowSegmentData	gps_traffic_density
elevation_m[]	Waypoint Snap API (batch)	road_grade / lookahead
grade_rad[]	Computed from elevation	gps_lookahead_grade

P0.6 · Traffic Flow Integration

```
def get_traffic_flow(segment_midpoint, api_key, zoom=10):
    lat, lon = segment_midpoint
    url = ('https://api.tomtom.com/traffic/services/4'
          f'/flowSegmentData/absolute/{zoom}/json')
    r = requests.get(url, params=dict(key=api_key,
    point=f'{lat},{lon}'), timeout=8)
    jam = r.json()['flowSegmentData'].get('jamFactor', 0)
    return jam / 10.0 # normalise to [0,1]
```

P0.7 · Speed Profile → Drive Cycle Conversion

In v3.1, `tomtom_route_to_cycle()` is the **ONLY** drive cycle generator. No CSV fallback exists. This function replaces WLTC/FTP-75/US06 entirely. The 1 Hz speed array it produces is the sole training signal for the RL agent.

```
def tomtom_route_to_cycle(segments: List[RouteSegment]) -> np.ndarray:
    """SOLE drive cycle source in v3.1 – no CSV fallback."""
    speeds = []
    for seg in segments:
        v_ms = min(seg.speed_limit_kmh, 130) / 3.6 * (1 - 0.4 * seg.traffic_density)
        n_steps = max(1, int(seg.length_m / max(v_ms, 0.5))) # 1 Hz
        speeds.extend([v_ms] * n_steps)
    return np.array(speeds, dtype=np.float32)
```

Phase 0 — Checkpoints

- ✓ TomTom API key set in .env — `fetch_tomtom_route()` returns valid GeoJSON
- ✓ Route segments cached to `gps/cache/*.json` — no redundant API calls
- ✓ Traffic flow `jam_factor` fetched per segment — `traffic_density` in [0, 1]
- ✓ Elevation profile fetched via Waypoint Snap API — `grade_rad` computed
- ✓ FRC road class mapped to `segment_type` 0/1/2
- ✓ `tomtom_route_to_cycle()` produces valid float32 1 Hz array (NO CSV used)
- ✓ Streamlit: Origin + Destination → route rendered on Folium map
- ✓ Route JSON loadable by THSEnv — `obs[5:8]` non-zero and change at boundaries

1 · Introduction & Problem Statement

The Toyota Hybrid System II (THS-II), deployed in the Prius Gen 3 (ZVW30), uses a planetary power-split device (PSD) allowing ICE, MG1, and MG2 to operate simultaneously or independently. This architecture makes energy management a complex multi-dimensional optimisation problem poorly served by static rule tables.

v3.1 removes all pre-recorded standard drive cycles (WLTC, FTP-75, US06). The RL agent is trained exclusively on real-world driving cycles generated on-the-fly from TomTom API routes — the user supplies Origin/Destination in the Streamlit dashboard, the backend fetches the route, and the RL agent trains and evaluates on the resulting TomTom speed profile.

ID	Objective	Status
OBJ-01	Wrap modeling1.py inside Gymnasium (8-dim obs, 4 actions) — TomTom route_cache mandatory	Mandatory
OBJ-02	Train PPO to select optimal drive mode per TomTom road segment	Mandatory
OBJ-03	Minimise CO2 emissions, fuel consumption, and total energy	Mandatory
OBJ-04	Maximise HV battery lifetime — minimise depth of discharge cycling	Mandatory
OBJ-05	Maintain SOC within $\pm 5\%$ of 60% charge-sustaining reference	Mandatory
OBJ-06	[v3.1] Generate ALL driving cycles from TomTom O/D pairs — no CSV	Mandatory
OBJ-07	Export trained policy to ONNX; validate on Raspberry Pi 4	Mandatory
OBJ-08	Streamlit dashboard — live route map, CO2, fuel, energy, SOC telemetry	Mandatory
OBJ-09	CARLA live co-simulation	Optional D6
OBJ-10	CAN Bus Hardware-in-the-Loop on RPi 4	Optional D7

2 · State of the Art

2.1 · Rule-Based EMS

Early EMS uses deterministic switching rules (charge-depleting / charge-sustaining). Computationally cheap but unable to account for route topology or traffic context.

2.2 · Optimal Control Methods

Dynamic Programming yields globally optimal policies over known drive cycles (acausal). ECMS reduces cost with co-state estimation. MPC adds rolling prediction horizon. None adapt to real-world route variability in real-time.

2.3 · Learning-Based EMS — This Work

PPO-based agents demonstrate 12–15% fuel reductions on standard cycles with SOC deviations below 3%. v3.1 distinguishing contributions: (1) TomTom real-route driving cycles are the **sole** training source; (2) multi-objective reward targeting CO2, fuel, energy and battery lifetime; (3) low-cost HIL pipeline exporting to ONNX for RPi 4.

2.4 · Prius Gen 3 Drive Mode Architecture

Mode	Activation	Behaviour Summary	EMS Effect
NORMAL	Default (no button)	Balanced ICE/motor split, standard regen	Baseline SOC target 60%
ECO	ECO button	Softened throttle, A/C reduced, lower EV threshold	Prefers low ICE load, maximises regen
PWR	PWR button	Aggressive throttle, higher MG2 boost	Relaxed SOC target, responsive torque
EV	EV button	Pure electric below 72 km/h, SOC $\geq 45\%$	ICE disabled unless SOC drops or speed exceeds cap

NOTE: SPORT mode is absent on the Prius Gen 3. Action space is strictly Discrete(4): 0=EV, 1=ECO, 2=NORMAL, 3=PWR. Value 4 must never appear anywhere in the codebase.

3 · System Architecture — Phase Overview

Phase	Description	Key Output
Phase 0 [MANDATORY]	TomTom API → route segments with traffic + elevation	route_cache.json + speed profile (SOLE cycle source)
Day 1 — Setup	Install stack, verify physics engine modeling1.py	Python env, modeling1.py KPIs confirmed
Day 2 — TomTom Env	Gymnasium env with TomTom route_cache (mandatory)	THSEnv validated, rule-based baseline on TomTom routes
Day 3 — Training	PPO training exclusively on TomTom routes (5 route types)	SB3 checkpoint on TomTom data only
Day 4 — Eval+ONNX	SIL validation on TomTom routes, ONNX export, Dashboard	ths_policy.onnx + Streamlit CO2 dashboard
Day 5 — GPS Fine-tune	TomTom GPS enrichment, pre-trip planner, fine-tuning	ths_agent_gps.zip + pre_trip_plan.json
Day 6 — CARLA [Opt]	RL mode output replaces rule selector in CARLA	Live traffic validation
Day 7 — CAN HIL [Opt]	ONNX policy on RPi 4 via CAN Bus 2.0B 500 kbit/s	Embedded validation

5 · Reinforcement Learning Design

5.1 · Multi-Objective MDP Formulation

Objective	Reward Term	Weight	Rationale
Minimise fuel	- fuel_rate_gs	×1.0	Direct fuel burn per timestep
Minimise CO2	- 0.00236 × fuel_rate_gs × 1000	×2.0	CO2 = 2.36 kg/L petrol; proportional to fuel
Minimise energy	- p_batt_kw × 0.3	×0.5	Penalise unnecessary HV bus power draw
Battery lifetime	- 5.0 × max(0, SOC-0.60 - 0.05) ²	×10.0	Penalise excursions beyond ±5% of 60% SOC
Regen bonus	+ 0.5 × max(0, -p_batt_kw)	×0.5	Reward regenerative braking capture
Grade anticipation	+ 0.3 × EV_on_flat, - 0.2 × EV_on_grade	GPS	Penalise EV on steep uphill (battery drain)

5.2 · State Space (8-dimensional)

Index	Variable	Unit	Source	Normalisation
obs[0]	velocity	m/s	TomTom speed profile	÷ 30
obs[1]	SOC	[0,1]	state.soc_pct / 100	direct
obs[2]	road_grade	rad	TomTom grade_rad per segment	÷ 0.3
obs[3]	segment_type	{0,1,2}	TomTom FRC → 0/1/2	÷ 2
obs[4]	acceleration	m/s ²	Δspeed/dt	÷ 5
obs[5]	gps_lookahead_grade	rad	next segment grade_rad	÷ 0.3, clip ±1
obs[6]	gps_traffic_density	[0,1]	TomTom jam_factor / 10	direct
obs[7]	gps_dist_to_next_seg	m	segment boundary distance	÷ 1000

NOTE: All 8 observation dimensions are populated from TomTom route data. obs[5:8] must be non-zero and change at segment boundaries — this is verified in the Phase 0 and Day 2 checklists.

5.3 · Evaluation Metrics

Metric	Definition	Target
Fuel savings (%)	RL vs rule-based on TomTom test route	> 5%
CO2 savings (%)	RL CO2 vs NORMAL fixed-mode (TomTom route)	> 5%
Total energy (kWh/km)	(fuel energy + ΔHV battery) / km	< NORMAL on TomTom
Battery DoD cycles	SOC excursions > 5% from 60%	< rule-based
SOC RMSE	RMSE of SOC from 60% over episode	< ±5%
GPS anticipation rate	% grade events with mode set before segment	> 60%
ONNX latency (PC)	sess.run() on PC (8-dim input)	< 2 ms
ONNX latency (RPi)	sess.run() on RPi 4	< 10 ms

DAY 1

1

Environment Setup & Physics Verification

Prerequisite: Phase 0 complete

v3.1 REQUIREMENT: Phase 0 (TomTom route pipeline) must be complete before Day 1. There are no CSV drive cycles in v3.1 — TomTom routes are the only cycle source.

Part 1A · Python Stack Installation

- **Create virtual environment (Python 3.11)** and activate.
- **RL stack:** stable-baselines3[extra] gymnasium==0.29 torch onnxruntime tensorboard
- **Data/viz stack:** numpy pandas matplotlib plotly streamlit folium streamlit-folium
- **TomTom/GPS stack:** requests geopy scipy shapely python-dotenv
- **Optional:** python-can (CAN HIL Day 7), carla (CARLA Day 6)
- Freeze: `pip freeze > requirements.txt`

Part 1B · Project Directory Structure

Path	Description
gps/	TomTom route fetcher, traffic, segmenter, pre-trip planner
gps/cache/	Pre-computed route JSON cache files (MD5-named)
env/	ths_env.py — TomTom route_cache MANDATORY (no CSV fallback)
env/drive_cycles/	REMOVED in v3.1 — directory no longer needed
training/	PPO training script, TomTom-based baseline, ONNX export
eval/	SIL evaluation on TomTom routes, metrics, plots
models/	SB3 checkpoints + ONNX models
app/	Streamlit dashboard with TomTom route input + CO2 telemetry
hil/	CAN Bus PC-side and RPi controller scripts [Optional D7]
carla/	CARLA RL bridge subclass [Optional D6]
modeling1.py	THS-II physics engine — project root, DO NOT MODIFY

Part 1C · Physics Verification

- Run `python modeling1.py --standalone` — confirm KPI values print, zero errors.
- Inspect DriveMode enum: confirm ECO, NORMAL, PWR, EV present. Confirm SPORT absent.
- Document WLTC physics baseline removed — TomTom route will serve as the Day 3 training baseline.

Day 1 — Checkpoints

- ✓ `python modeling1.py --standalone` → KPIs printed, zero errors
- ✓ DriveMode enum has EV/ECO/NORMAL/PWR — SPORT absent confirmed
- ✓ All Python imports succeed (SB3, Gymnasium, PyTorch, ONNX, Streamlit, TomTom requests)
- ✓ TomTom test fetch: `fetch_tomtom_route()` returns valid route JSON
- ✓ `env/drive_cycles/` directory NOT present (removed in v3.1)

DAY 2

2

TomTom Gymnasium Environment

TomTom route_cache is MANDATORY — no CSV fallback

Part 2A · Implementing env/ths_env.py — TomTom Only

BREAKING CHANGE: In v3.1, THSEnv does NOT accept cycle="WLTC", "FTP75", or "US06". route_cache is a REQUIRED parameter. The environment raises ValueError if route_cache is None.

- **__init__:** Accepts route_cache (required path to TomTom JSON), dt=0.1. Raises ValueError if route_cache is None.
- **observation_space:** Box(8,) low=-1, high=1, float32.
- **action_space:** Discrete(4) — 0=EV, 1=ECO, 2=NORMAL, 3=PWR. No action 4.
- **reset():** Instantiate THSIIController. Load TomTom speed profile via tomtom_route_to_cycle(). Reset idx, distance_m, speed.
- **step():** Map action → DriveMode. Call ems.step(). Compute multi-objective reward. Advance idx.
- **_obs():** 8-element normalised vector. All GPS dims (obs[5:8]) populated from TomTom segments.

```
class THSEnv(gym.Env):
    """v3.1 — TomTom route_cache MANDATORY, no CSV fallback."""
    def __init__(self, route_cache: str, dt: float = 0.1):
        if route_cache is None:
            raise ValueError('route_cache is required in v3.1 — no CSV fallback exists.')
        self.route_segments = load_tomtom_cache(route_cache)
        self.speed_profile = tomtom_route_to_cycle(self.route_segments)
        self.observation_space = spaces.Box(-1., 1., shape=(8,), dtype=np.float32)
        self.action_space = spaces.Discrete(4) # EV/ECO/NORMAL/PWR only
        self.dt = dt
```

Part 2B · Multi-Objective Reward Implementation

```
def _compute_reward(self, out, action):
    soc = out['soc']
    fuel_gs = out['fuel_rate_gs']
    p_batt = out['p_batt_kw']
    r_fuel = -fuel_gs
    r_co2 = -2.0 * fuel_gs * (2360 / 750)
    r_energy = -0.5 * abs(p_batt)
    excess = max(0.0, abs(soc - 0.60) - 0.05)
    r_life = -10.0 * excess ** 2
    r_regen = 0.5 * max(0.0, -p_batt)
    r_gps = self._grade_anticipation_bonus(action, out)
    return r_fuel + r_co2 + r_energy + r_life + r_regen + r_gps
```

Part 2C · Rule-Based Baseline on TomTom Routes (training/baseline_rule.py)

- EV if $v < 5$ km/h AND $soc \geq 0.45$; ECO if $v < 15$ km/h; NORMAL if $v < 80$ km/h; PWR otherwise.
- Baseline runs on the SAME TomTom route_cache as the RL agent — no CSV fallback.
- Record: total fuel (g), total CO2 (g), total energy (kWh/km), DoD cycle count, SOC trajectory.
- PPO must exceed rule-based by $\geq 5\%$ on fuel AND CO2 on Day 4.

Day 2 — Checkpoints

- ✓ THSEnv(route_cache=None) raises ValueError — confirmed
- ✓ THSEnv(route_cache="gps/cache/test.json") loads correctly
- ✓ check_env(THSEnv(route_cache="...")) → zero warnings
- ✓ action_space is Discrete(4) — no action 4 anywhere
- ✓ Multi-objective reward has 6 terms — confirmed in step() output
- ✓ obs[5:8] non-zero and change at segment boundaries
- ✓ Rule-based baseline runs on TomTom route — fuel/CO2/energy recorded

DAY 3

3

PPO Training on TomTom Routes

All training episodes use TomTom real-route speed profiles

Part 3A · TomTom Training Episode Pool

Create a pool of 5 representative TomTom routes covering diverse driving conditions. Each episode `reset()` randomly selects one route from the pool. **No WLTC, FTP-75, or US06 CSV cycles are used at any point.**

Route ID	Description	Characteristics	Key RL Challenge
R01	Flat urban (Paris 5ème)	High jam_factor, low speed, frequent stops	EV/ECO mode selection, regen
R02	Hilly rural (Aveyron)	Variable grade, low traffic, medium speed	Grade anticipation, SOC management
R03	Steep highway climb (Col de la Croix-de-Fer)	High grade_rad, highway speed, low jam	Avoid EV on grade, PWR timing
R04	Mixed suburban (Lyon banlieue)	Mixed FRC, medium traffic, varied speed	Mode transitions, battery lifetime
R05	Downhill coastal (Nice Corniche)	Negative grade, scenic speed limits	Regen bonus, SOC upper guard

Part 3B · PPO Training Script (training/train_ppo.py)

```
from stable_baselines3 import PPO
from stable_baselines3.common.env_util import make_vec_env
from stable_baselines3.common.callbacks import EvalCallback
import random, os

ROUTE_POOL = [
    'gps/cache/r01_paris_urban.json',
    'gps/cache/r02_aveyron_rural.json',
    'gps/cache/r03_col_highway.json',
    'gps/cache/r04_lyon_suburban.json',
    'gps/cache/r05_nice_coastal.json',
]

def make_env():
    cache = random.choice(ROUTE_POOL) # random route each episode
    return THSEnv(route_cache=cache)

vec_env = make_vec_env(make_env, n_envs=4)
eval_env = THSEnv(route_cache=ROUTE_POOL[0]) # fixed eval route

model = PPO('MlpPolicy', vec_env, verbose=1,
            learning_rate=3e-4, n_steps=2048, batch_size=64,
            n_epochs=10, gamma=0.99, gae_lambda=0.95,
            clip_range=0.2, ent_coef=0.01, tensorboard_log='./runs/')

cb = EvalCallback(eval_env, best_model_save_path='models/',
                 eval_freq=50_000, n_eval_episodes=5, deterministic=True)

model.learn(total_timesteps=500_000, callback=cb)
model.save('models/best_model')
```

Day 3 — Checkpoints

- ✓ All 5 TomTom routes cached in `gps/cache/` — no CSV files present
- ✓ PPO training starts — TensorBoard `ep_rew_mean` rising by step 100k
- ✓ `models/best_model.zip` exists after EvalCallback fires at 50k steps
- ✓ Training log shows route IDs cycling (not CSV cycle names)
- ✓ No `FileNotFoundError` on `WLTC.csv` or `FTP75.csv` (they do not exist)
- ✓ 500k timesteps complete — `models/best_model.zip` saved

DAY 4

4

Evaluation, ONNX Export & Dashboard

SIL validation on TomTom routes · ONNX export · Streamlit CO2 dashboard

Part 4A · Software-in-the-Loop Evaluation on TomTom Routes

- Load models/best_model.zip. Run deterministic=True on all 5 TomTom routes. Average ≥ 3 runs per route.
- Record per route: total fuel (g), total CO2 (g), energy (kWh/km), DoD cycle count, SOC RMSE, mode histogram.
- Compare RL vs rule-based AND vs NORMAL fixed-mode — all on TomTom routes. Fuel and CO2 savings must exceed 5%.
- Generate plots: SOC trace, CO2 curve, fuel bar chart, energy comparison, mode histogram, battery DoD count.

Part 4B · ONNX Export

```
import torch
from stable_baselines3 import PPO

model = PPO.load('models/best_model')
obs_ex = torch.zeros(1, 8) # 8-dim TomTom observation

torch.onnx.export(
    model.policy, obs_ex, 'models/th_s_policy.onnx',
    opset_version=17,
    input_names=['obs'], output_names=['action_logits'],
    dynamic_axes={'obs': {0: 'batch'}}
)
# Verify: target ~180-220 KB, 4-logit output, < 2 ms on PC
```

Part 4C · Streamlit Dashboard — v3.1 Extended

- **Sidebar:** Origin + Destination text fields, "Generate Route" button, agent mode (RL PPO / Rule-based).
- **Live telemetry:** SOC, cumulative fuel (g), cumulative CO2 (g), energy (kWh/km), battery DoD counter.
- **CO2 gauge:** Real-time kg CO2 vs NORMAL mode baseline comparison (both on TomTom route).
- **GPS map (Folium):** Route polyline coloured by recommended mode. Traffic density heatmap.
- **Summary table:** Total fuel (g), CO2 savings (%), energy savings (%), SOC RMSE, DoD cycles.
- **Mode donut:** 4 slices — EV / ECO / NORMAL / PWR with % labels.

Day 4 — Checkpoints

- ✓ RL fuel savings on TomTom routes > 5% vs NORMAL mode baseline
- ✓ RL CO2 savings on TomTom routes > 5% vs NORMAL mode baseline
- ✓ Battery DoD cycle count < rule-based baseline (TomTom route)
- ✓ models/th_s_policy.onnx exists (~180-220 KB), 4-logit output, < 2 ms on PC
- ✓ Streamlit dashboard: CO2 gauge, energy panel, GPS Folium map render correctly
- ✓ Git tag v3.1-sil committed with all deliverables

DAY 5

5

GPS Route Fine-Tuning & Predictive EMS

GPS-enriched fine-tuning · Pre-trip mode planner · CO2 estimate

Day 5 fine-tunes the Day 3 checkpoint using GPS-enriched TomTom episodes that emphasise grade lookahead and traffic density. The pre-trip mode planner generates a recommended mode sequence before departure.

Part 5A · TomTom-Enriched THSEnv (GPS variant)

```
class GPSWrappedEnv(THSEnv):
    """Randomly selects one of the 5 TomTom routes each episode."""
    def __init__(self, route_pool: list, dt=0.1):
        self.route_pool = route_pool
        super().__init__(route_cache=random.choice(route_pool), dt=dt)

    def reset(self, **kwargs):
        self.route_cache = random.choice(self.route_pool)
        self.route_segments = load_tomtom_cache(self.route_cache)
        self.speed_profile = tomtom_route_to_cycle(self.route_segments)
        return super().reset(**kwargs)
```

Part 5B · GPS-Enriched Fine-Tuning

- Fine-tune from Day 3 best checkpoint: 300,000 additional timesteps using GPSWrappedEnv.
- Save as models/th_s_agent_gps.zip.
- Export new ONNX: models/th_s_policy_gps.onnx — verify 4-logit output.

Part 5C · Pre-Trip Mode Sequence Generator

- Load models/th_s_agent_gps.zip. For each RouteSegment, build mock 8-element obs from segment features.
- Call model.predict(obs, deterministic=True) → recommended mode per segment.
- Output: (segment_id, start_km, end_km, recommended_mode, grade_rad, traffic_density, expected_CO2_g).
- Save to gps/pre_trip_plan.json. Render in Streamlit as colour-coded route table.

Part 5D · Expected CO2 Pre-trip Estimate

```
def estimate_trip_co2(segments, model):
    total_co2_g = 0.0
    for seg in segments:
        obs = build_obs_from_segment(seg)
        action, _ = model.predict(obs, deterministic=True)
        v_ms = seg.speed_limit_kmh / 3.6 * (1 - 0.4 * seg.traffic_density)
        fuel_gs_est = road_load_fuel_estimate(v_ms, action, seg.grade_rad)
        total_co2_g += fuel_gs_est * (2360 / 750) * (seg.length_m / max(v_ms, 0.1))
    return total_co2_g # grams CO2 for full trip
```

Day 5 — Checkpoints

- ✓ GPSWrappedEnv selects routes randomly — confirmed in reset() log
- ✓ GPS-fine-tuned model th_s_agent_gps.zip saved with Discrete(4) action space
- ✓ th_s_policy_gps.onnx has 4-logit output — verified with ONNX Runtime
- ✓ Pre-trip plan JSON with EV/ECO/NORMAL/PWR + expected_CO2_g per segment
- ✓ GPS anticipation rate > 60% on validation TomTom route
- ✓ Streamlit GPS map shows colour-coded pre-trip plan with CO2 estimate

DAY 6

6

CARLA Live Co-Simulation [Optional]

Override `_select_drive_mode()` with GPS-fine-tuned RL policy

Override `_select_drive_mode()` in a `RLBridgeSimulation` subclass. Load `ths_agent_gps.zip`. Build 8-element obs per CARLA tick. Map actions 0–3 to `DriveMode`. Log mode, SOC, speed, CO2 rate, CARLA location to `carla_rl_log.csv`.

Day 6 — Optional Checkpoints

- [Optional] CARLA server connects, vehicle spawns in Town03
- [Optional] `_select_drive_mode()` maps actions 0–3 to EV/ECO/NORMAL/PWR only
- [Optional] `carla_rl_log.csv` includes `co2_rate_gs` and `energy_kw` columns
- [Optional] No Python exceptions during a full 5-minute CARLA run

DAY 7

7

CAN Bus Hardware-in-the-Loop [Optional]

ONNX policy on RPi 4 via CAN Bus 2.0B 500 kbit/s

Mount MCP2515 CAN HAT on RPi 4. Flash Raspberry Pi OS 64-bit. CAN frame 0x100 (PC→RPi): velocity, SOC, road_grade, segment_type, acceleration. CAN frame 0x200 (RPi→PC): mode_action 0–3 only (SPORT=4 deprecated), latency_ms, frame_counter, checksum.

NOTE: mode_action byte encodes only values 0–3 (EV/ECO/NORMAL/PWR). Value 4 must NEVER appear.

Day 7 — Optional Checkpoints

- [Optional] CAN loopback test passes
- [Optional] ths_policy_gps.onnx benchmark on RPi < 5 ms per call (8-dim input)
- [Optional] 0x200 mode_action byte encodes only values 0–3
- [Optional] Full TomTom-route HIL episode completes without CAN error
- [Optional] Mean HIL latency < 5 ms, max < 10 ms; CAN frame error rate < 0.1%
- [Optional] RPi peak RAM < 200 MB, CPU < 30%

§ 13 · Project Deliverables

ID	Path	Description	Status
SW-00a	gps/route_fetcher_tomtom.py	TomTom Routing + Traffic + Elevation fetch	Phase 0
SW-00b	gps/segmenter_tomtom.py	RouteSegment builder from TomTom JSON	Phase 0
SW-00c	gps/cache/*.json	Cached TomTom route files (5 routes × MD5-named)	Phase 0
SW-01	env/ths_env.py	Gymnasium env (8-dim obs, 4 actions, TomTom mandatory)	Mandatory
SW-02	modeling1.py	THS-II v4 physics engine — DO NOT MODIFY	Mandatory
SW-03	env/drive_cycles/	REMOVED in v3.1 — TomTom routes replace all CSV cycles	DELETED
SW-04	training/train_ppo.py	PPO training on 5 TomTom routes (no CSV)	Mandatory
SW-05	training/baseline_rule.py	Rule-based EMS on TomTom routes (CO2+fuel+energy)	Mandatory
SW-06	training/export_onnx.py	PyTorch policy → ONNX (8-dim in, 4-logit out)	Mandatory
SW-07	eval/sil_evaluate.py	SIL validation — fuel, CO2, energy, lifetime (TomTom)	Mandatory
SW-08	app/streamlit_dashboard.py	Dashboard: TomTom O/D input, CO2 gauge, GPS Folium map	Mandatory
SW-09	models/best_model.zip	Best SB3 PPO checkpoint (trained on TomTom routes)	Mandatory
SW-10	models/ths_policy.onnx	ONNX model (4-logit output, 8-dim TomTom obs)	Mandatory
SW-14	models/ths_agent_gps.zip	GPS-fine-tuned SB3 checkpoint (TomTom enriched)	Day 5
SW-15	models/ths_policy_gps.onnx	GPS-aware ONNX model (8-dim, 4-logit)	Day 5
SW-13	gps/pre_trip_planner.py	Pre-trip CO2 estimate + mode sequence generator	Day 5
SW-16	carla/carla_rl_bridge.py	CARLA ↔ RL bridge (4-mode, CO2 logging)	Opt D6
SW-17	hil/pc_side_can.py	PC CAN state transmitter (4-mode 0x200)	Opt D7
SW-18	hil/rpi_controller.py	RPi ONNX inference + CAN responder	Opt D7

v3.1 DELETION: SW-03 (env/drive_cycles/ with WLTC.csv, FTP75.csv, US06.csv) is DELETED in v3.1. Any reference to CSV drive cycles in the codebase is an error.

§ 14 · Conclusion & Future Work

14.1 · Summary

v3.1 delivers a complete, reproducible pipeline for learning-based Energy Management in a Toyota THS-II hybrid (Prius Gen 3, ZVW30) using TomTom real-world routes as the **sole training and evaluation data source**. Key advances: (1) all CSV drive cycles (WLTC, FTP-75, US06) removed — TomTom routes are mandatory; (2) the day order is fully logical (Setup → TomTom Env → Training → Eval → GPS Fine-tune → optional CARLA/CAN); (3) the multi-objective reward jointly minimises CO2, fuel, energy, and battery lifetime degradation; (4) the Streamlit dashboard exposes CO2 savings and pre-trip energy estimates.

14.2 · Expected Results

Parameter	Value
Fuel savings (SIL)	5–15% vs NORMAL fixed-mode on TomTom routes
CO2 savings (SIL)	5–15% vs NORMAL (proportional to fuel, TomTom routes)
Energy savings	Additional 2–5% from predictive mode anticipation (GPS)
Battery DoD cycles	< rule-based baseline across all TomTom test routes
SOC deviation	< ±5% RMSE from 60% across all test routes
GPS anticipation rate	> 60% of grade transitions pre-selected correctly
ONNX latency (PC)	< 2 ms per call (8-dim input, 4-logit output)
ONNX latency (RPi 4)	< 5 ms per call [Optional D7]

14.3 · Future Work

ID	Topic	Description	Effort
FW-01	Full CARLA Emergent Traffic	Replace prescribed cycles with dynamic CARLA + TomTom GPS	High
FW-02	Emissions Sensor Fusion	Add NOx, HC to reward — requires exhaust model extension	Medium
FW-03	Live TomTom Rerouting	Update route mid-trip on traffic change — online RL	High
FW-04	THS-V Migration	Port modeling1.py to THS-V (Prius Gen 5) parameters	High
FW-05	dSPACE HIL	Automotive-grade HIL certification platform	High
FW-06	SAC / TD3 Comparison	Benchmark off-policy algorithms for sample efficiency	Low
FW-07	Federated Fleet Learning	Federated RL across multiple vehicles — shared backbone	Research
FW-08	Hierarchical RL	High-level TomTom planner + low-level torque controller	Research